30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 1

Activity Lifecycle, State & Process Death



Esmaeeil Moradi
@esmaeeil-moradi700

What's inside Day 1

Every answer is written the way a senior interviewer wants to hear it:

- **Answer** — the mechanism, stated precisely
- **Interview trap** — where most candidates slip
- **Follow-up** — what you'll be asked next

2026 baseline: targetSdk 36 (Android 16), Kotlin, Coroutines & Flow, Jetpack Compose, current AndroidX Lifecycle and Navigation. Views and Fragments are covered where interviews still ask about them.

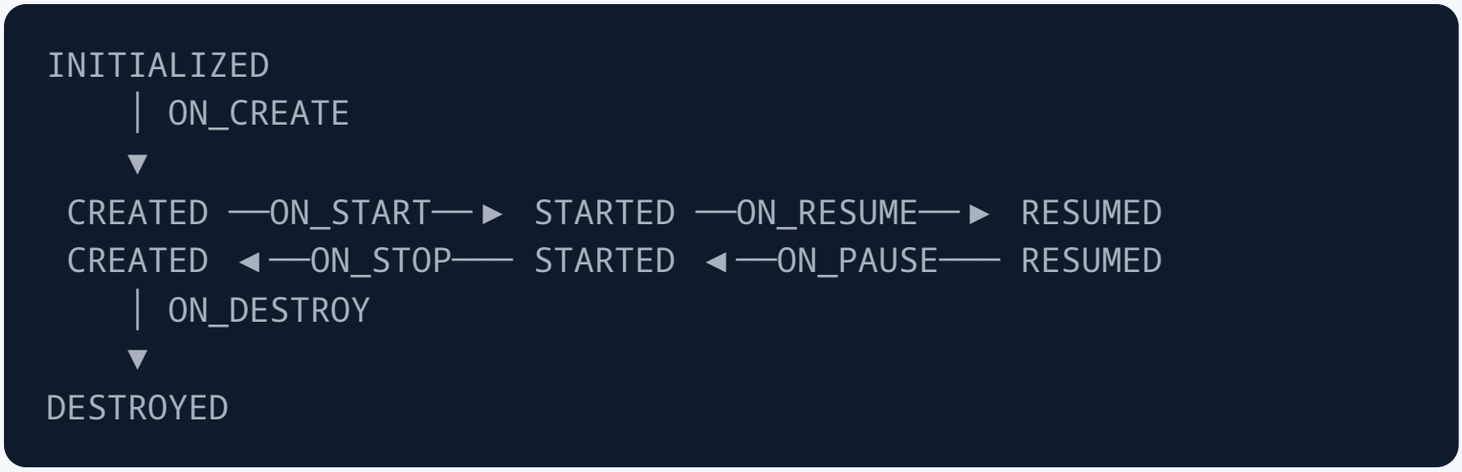
Level	Questions
Junior	Callbacks · Configuration changes · Saving state
Mid	Leaks · `onStop` vs `onDestroy` · Navigation & results · Tasks
Senior	Lifecycle-aware components · Startup · Multi-screen restoration
Architect	Fleet monitoring · Real-time streams · Process survival



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: states, not callbacks

A `Lifecycle` has **5 states**. Events move it between them:



- `CREATED` — exists, not visible (after `onCreate`, and again after `onStop`)
- `STARTED` — visible, maybe behind a translucent activity or in multi-window
- `RESUMED` — visible and interactive
- `DESTROYED` — terminal, no more events

WHY IT MATTERS

Modern APIs — `repeatOnLifecycle`, `collectAsStateWithLifecycle`, `LifecycleStartEffect` — are expressed in **states**. Think "work while at least STARTED", not "start in `onResume`, stop in `onPause`".



Esmaeeil Moradi
@esmaeeil-moradi700

Junior

Callbacks · Configuration changes · Saving state



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Walk me through the Activity lifecycle

Callback	State after	What belongs here
`onCreate`	CREATED	`setContent` / inflate, obtain the ViewModel, read `savedInstanceState`
`onStart`	STARTED	Now visible. Start UI-bound work (collect flows)
`onResume`	RESUMED	Interactive. Exclusive resources: camera, high-rate sensors
`onPause`	STARTED	**Keep it fast.** Pause focus-bound work. Still visible in multi-window
`onStop`	CREATED	Not visible. Stop UI work, release heavy resources, persist drafts
`onDestroy`	DESTROYED	Final cleanup. **Not guaranteed to run**

`onRestart` runs before `onStart` when a stopped activity comes back to the foreground.



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Order matters: A starts B

```
A.onPause → B.onCreate → B.onStart → B.onResume → A.onStop
```

- B is created **before** A stops, so a slow `A.onPause` delays B's first frame.
- If B is translucent or dialog-themed, A stays **STARTED** — no `onStop`.
- Back from B: `B.onPause → A.onRestart → A.onStart → A.onResume → B.onStop → B.onDestroy`

INTERVIEW TRAP

An `AlertDialog` or `DialogFragment` does **not** pause the activity — it's just a window in the same activity. Only another *activity* (a dialog-themed one, or the runtime-permission prompt) triggers `onPause`.

INTERVIEW TRAP

Never put must-save logic only in `onDestroy`. When the system kills your process, `onDestroy` is simply not called.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. What happens on a configuration change?

By default the activity is **destroyed and recreated**, so resources reload for the new configuration:

```
onPause → onStop → onSaveInstanceState → onDestroy
            ── new instance ──
onCreate(bundle) → onStart → onRestoreInstanceState → onResume
```

- Triggers: rotation, window resize (multi-window, foldables, desktop windowing), locale, dark theme, font scale, density, keyboard.
- `isChangingConfigurations` is `true` in `onStop` / `onDestroy`.
- Since API 28, `onSaveInstanceState` runs **after** `onStop` (before API 28: before `onStop`).
- The `ViewModelStore` survives because `ComponentActivity` hands it to the new instance through `onRetainNonConfigurationInstance` — in memory, same process.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. What changed in 2026

ANDROID 16 (API 36)

On large screens (smallest width ≥ 600dp), apps targeting API 36 have `screenOrientation`, `resizeableActivity` and aspect-ratio limits **ignored** (games excepted). "Lock to portrait" is no longer a way to avoid recreation — correct state handling is mandatory.

INTERVIEW TRAP

`android:configChanges` avoids recreation only for the listed changes. It does **nothing** for process death, and you must re-read resources from the new `Configuration`. In Compose-only apps it can be a deliberate trade-off; in View-based apps it's usually a smell.

FOLLOW-UP

"Why does a ViewModel survive rotation but not process death?" — It's kept in memory and handed to the new instance. Process death wipes memory; only the saved-state `Bundle`, held by the system, comes back.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. How do you preserve state?

Classify the state first — the tool follows:

State	Example	Where it lives
UI element	scroll, text input, expanded card	`rememberSaveable` / View saved state
Screen	loaded list, loading / error	`ViewModel` exposing `StateFlow`
Keys to rebuild	item id, query, selected tab	`SavedStateHandle`
App data	user, cart, drafts	Room / DataStore (+ network)

Survives…	Rotation	Process death	User dismisses task
ViewModel field	**Yes**	**No**	**No**
Saved state (`Bundle`, `SavedStateHandle`, `rememberSaveable`)	**Yes**	**Yes**	**No**
Room / DataStore	**Yes**	**Yes**	**Yes**

"User dismisses" = Back that finishes the activity, swipe from Recents, force stop.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Save the key, re-derive the data

```
class SearchViewModel(
    private val savedStateHandle: SavedStateHandle,
    private val repo: SearchRepository,
) : ViewModel() {

    val query: StateFlow<String> =
        savedStateHandle.getStateFlow(KEY_QUERY, "")

    val results: StateFlow<List<Item>> = query
        .map { q -> repo.search(q) }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5_000),
            initialValue = emptyList(),
        )

    fun onQueryChange(value: String) {
        savedStateHandle[KEY_QUERY] = value
    }

    private companion object { const val KEY_QUERY = "query" }
}
```

```
// Compose: element-level UI state
var expanded by rememberSaveable { mutableStateOf(false) }
```

The query (a few bytes) is saved; the results (potentially large) are rebuilt from the repository after process death.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. The two traps everyone falls into

INTERVIEW TRAP

"onSaveInstanceState doesn't survive process death." — **Wrong.** Surviving system-initiated process death is exactly its job. It's discarded only when the **user** dismisses the task.

SIZE LIMIT

Saved state crosses Binder. The transaction buffer is about **1 MB, shared by the whole process**, so keep saved state small (tens of KB at most). Overflow ends in `TransactionTooLargeException`. Save ids, never lists or bitmaps.

FOLLOW-UP

"Where does `SavedStateHandle` get its data after process death?" — From the same saved-state `Bundle` the activity or back-stack entry saved. For an activity, its default values also include the `Intent` extras.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 1



Mid-Level

Leaks · onStop vs onDestroy · Navigation & results · Tasks



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Where do Activity leaks come from today?

A leak is an object with a **longer lifetime** holding a reference to one with a **shorter** lifetime: an Activity, a View, a Fragment's view.

Real-world sources in modern code:

- A singleton or `@Singleton` dependency holding an **Activity context** or a `View`
- A `ViewModel` holding a `View`, an `Activity`, or a lambda that captures them
- A Fragment keeping its view binding after `onDestroyView` — the Fragment outlives its view
- Coroutines in `GlobalScope` or in a custom scope nobody cancels
- Listeners registered with system services or SDKs and never removed
- `Handler.postDelayed` with a pending runnable that captures the activity



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Fix it by tying work to a lifetime

```
// Activity
lifecycleScope.launch {
    repeatOnLifecycle(Lifecycle.State.STARTED) {
        viewModel.uiState.collect(::render)
    }
}

// Fragment: the view's lifecycle, not the Fragment's
viewLifecycleOwner.lifecycleScope.launch { /* same as above */ }

// Compose
val state by viewModel.uiState.collectAsStateWithLifecycle()
```

- Structured concurrency: `viewModelScope`, `lifecycleScope`
- Long-lived objects get `applicationContext`, never an Activity
- Detect with **LeakCanary** in debug builds and heap dumps in the Memory Profiler
- `WeakReference` is a last resort — usually a sign the ownership design is wrong

INTERVIEW TRAP

`launchWhenStarted` is deprecated: it **suspends** the coroutine but keeps the upstream flow alive, so a location or socket producer keeps running in the background. `repeatOnLifecycle` **cancels** and restarts.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. onStop() vs onDestroy()

	`onStop()`	`onDestroy()`
Meaning	No longer visible	This instance is going away
What comes next	`onRestart`, `onDestroy`, **or a process kill with no callback**	Nothing
Reliable?	Last callback you can rely on before a kill	**No**
Do here	Persist drafts, stop UI work, release camera / players	Release what the instance owns
Useful check	—	`isFinishing` vs `isChangingConfigurations`

INTERVIEW TRAP

Android does **not** destroy individual activities to reclaim memory — it **kills the whole process**. "The system called `onDestroy` to free memory" is wrong: after a kill you get no callback at all, only a restore when the user returns.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Navigating and passing data

Rule: pass **identifiers**, not objects. The destination loads the object from the repository — one source of truth and tiny saved state.

```
// Start
startActivity(
    Intent(this, DetailActivity::class.java)
        .putExtra(EXTRA_ITEM_ID, itemId)
)

// Read. Better: in DetailViewModel,
// savedStateHandle[EXTRA_ITEM_ID] already contains the extras
val itemId = intent.getLongExtra(EXTRA_ITEM_ID, -1L)
```

- If you must pass an object: `@Parcelize` rather than `Serializable` (no reflection, faster).
- Android 13+ has typed `getParcelableExtra(key, Class)`; use `IntentCompat` to support older versions.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Single-activity: type-safe navigation

```
@Serializable data object Home
@Serializable data class Detail(val itemId: Long)

NavHost(navController, startDestination = Home) {
    composable<Home> {
        HomeScreen(
            onOpen = { id -> navController.navigate(Detail(id)) }
        )
    }
    composable<Detail> { DetailScreen() }
}

class DetailViewModel(handle: SavedStateHandle) : ViewModel() {
    private val itemId = handle.toRoute<Detail>().itemId
}
```

- Arguments are checked at compile time, and they live in the back-stack entry's saved state, so they survive process death.
- Safe Args solved the same problem for XML graphs and Fragments.
- **Navigation 3** is the newer Compose-first library: the back stack is plain state that you own.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Getting a result back

`startActivityForResult` / `onActivityResult` are deprecated. Use the Activity Result API:

```
class ProfileActivity : ComponentActivity() {
    private val viewModel: ProfileViewModel by viewModels()

    private val pickPhoto = registerForActivityResult(
        ActivityResultContracts.PickVisualMedia()
    ) { uri: Uri? -> uri?.let(viewModel::onPhotoPicked) }

    fun onChangePhotoClick() {
        pickPhoto.launch(
            PickVisualMediaRequest(
                ActivityResultContracts.PickVisualMedia.ImageOnly
            )
        )
    }
}
```

In Compose: `rememberLauncherForActivityResult(...)`. The photo picker needs no storage permission.

INTERVIEW TRAP

Register unconditionally during initialization (a property is fine). Registering after the lifecycle reaches STARTED — for example inside a click listener — throws `IllegalStateException`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Tasks, launch modes & onNewIntent

Mode / flag	Behavior
`standard`	A new instance every time
`singleTop` / `FLAG_ACTIVITY_SINGLE_TOP`	Reused only if already on **top** → `onNewIntent`
`singleTask`	One instance in its task; activities above it are cleared → `onNewIntent`
`singleInstancePerTask` (API 31)	Always the root of its task; can exist in several tasks
`singleInstance`	The only activity in its task
`NEW_TASK` + `CLEAR_TASK`	Classic "logout → fresh login" reset

INTERVIEW TRAP

In `onNewIntent`, `getIntent()` still returns the **original** intent unless you call `setIntent(intent)`. With `ComponentActivity` you can also use `addOnNewIntentListener`.

FOLLOW-UP

"What happens on Back from the root activity?" — Since Android 12, Back on a root launcher activity moves the task to the background instead of finishing it. The activity is kept (no `onDestroy`), so the next launch is a warm start.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 1



Senior

Lifecycle-aware components · Startup · Multi-screen restoration



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Design a lifecycle-aware resource manager

Goal: the resource (location, socket, sensor, player) runs **only while someone visible needs it**, with no start/stop code in screens.

Choose the scope:

- **Per screen** → expose a cold `Flow`; consumers collect while at least STARTED
- **Per app** → `ProcessLifecycleOwner`: foreground/background across all activities

Choose the states deliberately:

- **STARTED ↔ STOPPED** for anything the user can *see* — in multi-window, several activities are visible at once
- **RESUMED ↔ PAUSED** only for exclusive resources such as the camera. Since Android 10 several activities can be resumed at once; use `onTopResumedActivityChanged` for work that needs the single top-focused activity



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Per screen: callbackFlow

```
class LocationRepository(
    private val client: FusedLocationProviderClient,
) {
    @RequiresPermission(
        anyOf = [ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION]
    )
    fun updates(intervalMs: Long = 5_000) = callbackFlow<Location> {
        val request = LocationRequest.Builder(
            Priority.PRIORITY_BALANCED_POWER_ACCURACY, intervalMs
        ).build()

        val callback = object : LocationCallback() {
            override fun onLocationResult(result: LocationResult) {
                result.lastLocation?.let { trySend(it) }
            }
        }

        client.requestLocationUpdates(
            request, callback, Looper.getMainLooper()
        )
        awaitClose { client.removeLocationUpdates(callback) }
    }
}
```

The flow starts when collected and **cleans up in `awaitClose`** when the collector is cancelled — which `repeatOnLifecycle(STARTED)` does on `onStop`. No Activity reference, no manual start/stop.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Per app: ProcessLifecycleOwner

```
class AppForegroundObserver(
    private val socket: RealtimeSocket,
) : DefaultLifecycleObserver {
    override fun onStart(owner: LifecycleOwner) = socket.connect()
    override fun onStop(owner: LifecycleOwner) = socket.disconnect()
}

// Application.onCreate()
ProcessLifecycleOwner.get().lifecycle
    .addObserver(AppForegroundObserver(socket))
```

- `ProcessLifecycleOwner` dispatches `ON_STOP` with a short delay (~700 ms) after the last activity stops, so a rotation doesn't look like "app went to background". It never dispatches `ON_DESTROY`.
- Unit test the observer with `TestLifecycleOwner` from `lifecycle-runtime-testing`.

INTERVIEW TRAP

`@OnLifecycleEvent` is deprecated. Use `DefaultLifecycleObserver` or `LifecycleEventObserver`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Optimize startup — measure first

Type	Already in memory	Android vitals "excessive"
Cold	Nothing — process, `Application`, Activity all created	≥ 5 s
Warm	Process alive, Activity recreated (or restored from saved state)	≥ 2 s
Hot	Process and Activity alive, brought to front	≥ 1.5 s

- **TTID** (time to initial display): first frame — the `Displayed` line in Logcat
- **TTFD** (time to full display): screen actually usable — `reportFullyDrawn()`, or in Compose `ReportDrawnWhen { items.isNotEmpty() }`
- **Tools:** Macrobenchmark (lab and CI), Perfetto traces (Systrace is legacy), Android vitals (real users)



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. What actually moves the needle

1. **Baseline Profiles** (+ Startup Profiles): critical code paths are compiled ahead of time from the first launch. Google cites about 30% faster code execution on first launch.
2. **Don't block the first frame:** no network or disk on the main thread; show cached data or a skeleton.
3. **Lazy initialization:** `Lazy<T>` / `Provider<T>` in Hilt, `by inject()` in Koin. Replace SDK auto-init ContentProviders with App Startup, or init on demand.
4. **R8** (full mode is the default since AGP 8) and removing dead dependencies.
5. **SplashScreen API** (`core-splashscreen`): use `setKeepOnScreenCondition` only for a short, local check.
6. **StrictMode** in debug builds to catch main-thread disk and network access.
7. **Gate regressions:** run Macrobenchmark on a real device in CI.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Benchmark and profile in code

```
@RunWith(AndroidJUnit4::class)
class StartupBenchmark {
    @get:Rule val rule = MacrobenchmarkRule()

    @Test fun coldStartup() = rule.measureRepeated(
        packageName = "com.example.app",
        metrics = listOf(StartupTimingMetric()),
        iterations = 10,
        startupMode = StartupMode.COLD,
    ) {
        pressHome()
        startActivityAndWait()
    }
}

@RunWith(AndroidJUnit4::class)
class BaselineProfileGenerator {
    @get:Rule val rule = BaselineProfileRule()

    @Test fun generate() = rule.collect("com.example.app") {
        pressHome()
        startActivityAndWait()
        // + the critical user journey: open list, scroll, open detail
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. State restoration across many screens

- **One source of truth** (Room / cache). Screens re-derive their state; they don't keep private copies.
- **Each destination owns its ViewModel** and saves only *keys* in `SavedStateHandle`.
- **Let navigation save the back stack.** `NavController` and `FragmentManager` persist the back stack and each entry's saved state.
- Bottom navigation with **multiple back stacks**:

```
navController.navigate(tab.route) {
    popUpTo(navController.graph.findStartDestination().id) {
        saveState = true
    }
    launchSingleTop = true
    restoreState = true
}
```

- Fragments: observe with `viewLifecycleOwner`. `replace` + back stack saves the old view's state and frees its views; `show` / `hide` keeps every view in memory — use it only for cheap, frequently switched tabs.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Test it for real

Technique	What it simulates
Rotate / `ActivityScenario.recreate()`	Configuration change
Developer option "Don't keep activities"	Activity destruction — **process stays alive**
Background the app → `adb shell am kill <package>` → reopen from Recents	**Real process death**
Compose `StateRestorationTester`	`rememberSaveable` in tests

INTERVIEW TRAP

"Don't keep activities" does **not** simulate process death: singletons and in-memory caches survive, so the worst bugs stay hidden. Use `am kill` on a backgrounded app.

FOLLOW-UP

"How do you prevent regressions?" — An instrumented test per critical screen that recreates it from saved state, plus a manual `am kill` pass in the release checklist.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 1



Architect

Fleet monitoring · Real-time streams · Process survival



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Monitor lifecycle performance app-wide

1. In-app instrumentation — low overhead, sampled

- `ActivityLifecycleCallbacks` + `FragmentLifecycleCallbacks` → per-screen timings
- `androidx.tracing` sections → visible in Perfetto and Macrobenchmark
- **JankStats** → janky frames tagged with the current screen
- **`ApplicationExitInfo`** (API 30+) → *why* the previous process died

2. Transport — batch locally, upload with WorkManager, sample per release, no PII

3. Field + lab

- Field: Android vitals, Firebase Performance / Crashlytics, or your own backend
- Lab: Macrobenchmark on physical devices in CI, failing on P50/P90 regressions

4. Dashboards & alerts — sliced by app version, OS and device tier; alert on regressions during staged rollout



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Collection points

```
class ScreenTimingCallbacks(
    private val sink: MetricsSink,
) : Application.ActivityLifecycleCallbacks {

    private val started = HashMap<Int, Long>()
    private val Activity.key get() = System.identityHashCode(this)
    private fun now() = SystemClock.elapsedRealtime()

    override fun onActivityPreCreated(a: Activity, state: Bundle?) {
        started[a.key] = now()
    }

    override fun onActivityResumed(a: Activity) {
        val start = started.remove(a.key) ?: return
        sink.record(
            name = "create_to_resume",
            screen = a.javaClass.simpleName,
            millis = now() - start,
        )
    }
    // remaining callbacks: empty overrides
}
```

`create_to_resume` is a cheap proxy. For the real first frame, trust TTID/TTFD from Macrobenchmark and Android vitals.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Learn why the process died

```
fun reportPreviousExits(context: Context, store: ExitStore) {
    val am = context.getSystemService(ActivityManager::class.java)
    am.getHistoricalProcessExitReasons(context.packageName, 0, 10)
        .filter { it.timestamp > store.lastReportedTimestamp }
        .forEach { exit ->
            store.report(
                reason = exit.reason, // LOW_MEMORY, ANR, CRASH…
                importance = exit.importance, // fg or cached?
                pssKb = exit.pss,
                description = exit.description,
            )
        }
}
```

Low-memory kills of **cached** processes are normal. A spike after a release, or kills while **foreground**, is a real signal.

ANDROID 15+

`ProfilingManager` lets the app request heap dumps, stack sampling or system traces from real users' devices, rate-limited by the system — field traces instead of guesses.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Lifecycle-aware real-time data

A price screen needs a live WebSocket feed only while visible, without reconnecting on every rotation.

```
// Money is never a Double
data class Price(val symbol: String, val value: BigDecimal)

class PriceViewModel(
    repo: PriceRepository,
    savedStateHandle: SavedStateHandle,
) : ViewModel() {
    private val symbol: String =
        checkNotNull(savedStateHandle["symbol"])

    val state: StateFlow<PriceUiState> = repo.prices(symbol)
        .map<Price, PriceUiState> { PriceUiState.Live(it) }
        .retryWhen { cause, attempt ->
            val retry = cause is IOException
            if (retry) delay(backoffMs(attempt))
            retry
        }
        .catch { emit(PriceUiState.Error) }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5_000),
            initialValue = PriceUiState.Loading,
        )
}

// UI
val state by viewModel.state.collectAsStateWithLifecycle()
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Why this design wins

- **Rotation:** the new UI subscribes within 5 s, so `WhileSubscribed(5_000)` keeps the socket open — no reconnect.
- **Background:** collection stops at `onStop`; 5 s later the upstream is cancelled and `awaitClose` closes the socket.
- **Resilience:** exponential backoff on network errors; other errors become UI state instead of crashes.
- **Testable:** the repository is a plain `Flow` — test with a fake socket and Turbine, no lifecycle involved.
- **Correct money type:** `BigDecimal` (or minor units as `Long`), never `Double`.

WHEN DO YOU STILL WRITE A LIFECYCLEOBSERVER?

For APIs that aren't streams: camera and media players, third-party SDKs with `start()` / `stop()`, analytics screen tracking — and app-wide concerns via `ProcessLifecycleOwner`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Activity management vs. process survival

How the system decides:

- Process importance: **foreground → visible → perceptible / service → cached**
- `lmkd` (userspace, pressure-driven via PSI since Android 10) kills by `oom_score_adj` — **cached processes first**
- Modern Android also **freezes** cached processes so they use no CPU
- The system kills **processes**, not individual activities

What improves your odds:

- A small background footprint: drop caches and bitmaps in `onTrimMemory(TRIM_MEMORY_UI_HIDDEN)`
- No leaks — leaked activities mean a bigger heap and an earlier kill
- But **design as if you will be killed** — because you will be



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. The #1 process-death bug

User is on Checkout → app goes to background → process is killed → user returns.

Android restores **only the Checkout activity**, with its saved state. `Application.onCreate` runs again, but the screens **before** Checkout never ran in this process.

```
// BAD: SessionHolder was filled by LoginActivity.
// After process death it's empty -> crash.
val user = SessionHolder.user!!

// GOOD: re-derive from the source of truth (DataStore / Room)
val user: StateFlow<User?> = userRepository.currentUser
    .stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5_000),
        initialValue = null,
    )
```

Rule: every screen must be able to cold-start from its saved keys plus the repository. No screen may rely on in-memory state set by a previous screen.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Work that must outlive the UI

Need	Tool
Deferrable but guaranteed (sync, uploads)	**WorkManager** — persisted, retried, survives process death and reboot
Large user-started transfer (Android 14+)	**User-initiated data transfer job**
Ongoing and user-visible (playback, navigation, calls)	**Foreground service** with a declared type

Constraints worth naming in the interview:

- Android 12+: starting a foreground service **from the background** is blocked in most cases
- Android 14+: every foreground service needs a `foregroundServiceType` and the matching permission
- Android 15+: `dataSync` and `mediaProcessing` services are **time-limited** (6 hours per 24 hours)



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 1 · ARCHITECT

Day 1 — cheat sheet

- Think in **states**: do UI work while at least STARTED.
- `onPause` must be fast; `onStop` is the last reliable callback; `onDestroy` may never run.
- Dialogs don't pause an activity — other activities do.
- ViewModel survives rotation, **not** process death.
- Saved state survives process death, **not** user dismissal. Keep it tiny.
- Pass ids, not objects. Re-derive from the repository.
- `repeatOnLifecycle` / `collectAsStateWithLifecycle` — never `launchWhenX`.
- "Don't keep activities" ≠ process death. Test with `am kill`.
- Startup: TTID/TTFD, Baseline Profiles, Macrobenchmark in CI.
- Every screen must cold-start from saved keys + repository.

Day 2 is next → Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700